

Convolutional Neural Networks (CNN)

Week 3 & 4: CNN – Course Outline

➤ **Components of CNN**

- ✓ Fundamentals of image representation

➤ **Introduction to Convolutional Neural Networks**

- ✓ Layers in CNN

➤ **Implementation of CNN using Keras**

- ✓ Basic model

➤ **Standard CNN models:** AlexNet, VGGNet, GoogLeNet, ResNet, Inception, Transfer Learning

- ✓ Implementation of the models without transfer learning
- ✓ With transfer learning
- ✓ Ensemble model

Week 3

Week 4

Today's Session Overview

➤ **Components of CNN: Fundamentals and Preprocessing**

- ✓ Fundamentals of image representation
- ✓ Number of channels
- ✓ Why not use Fully Connected NN for Images?
- ✓ Data augmentation and how to store the data

Disclaimer:

The images used in this presentation are sourced from publicly available academic and internet resources.

They are used **solely for educational purposes** to support the understanding of key concepts in **Convolutional Neural Networks (CNNs)**.

Reference:

1. Ian Goodfellow, Yoshua Bengio, and Aaron Courville, *Deep Learning*, MIT Press, 2016. ✓
2. CS231n: Deep Learning for Computer Vision, Stanford University, Spring 2025. ✓
3. François Chollet, *Deep Learning with Python*, 2nd Edition, Manning Publications, 2021.

Image types



(a) RGB image



(b) Grayscale image



(c) Binary image

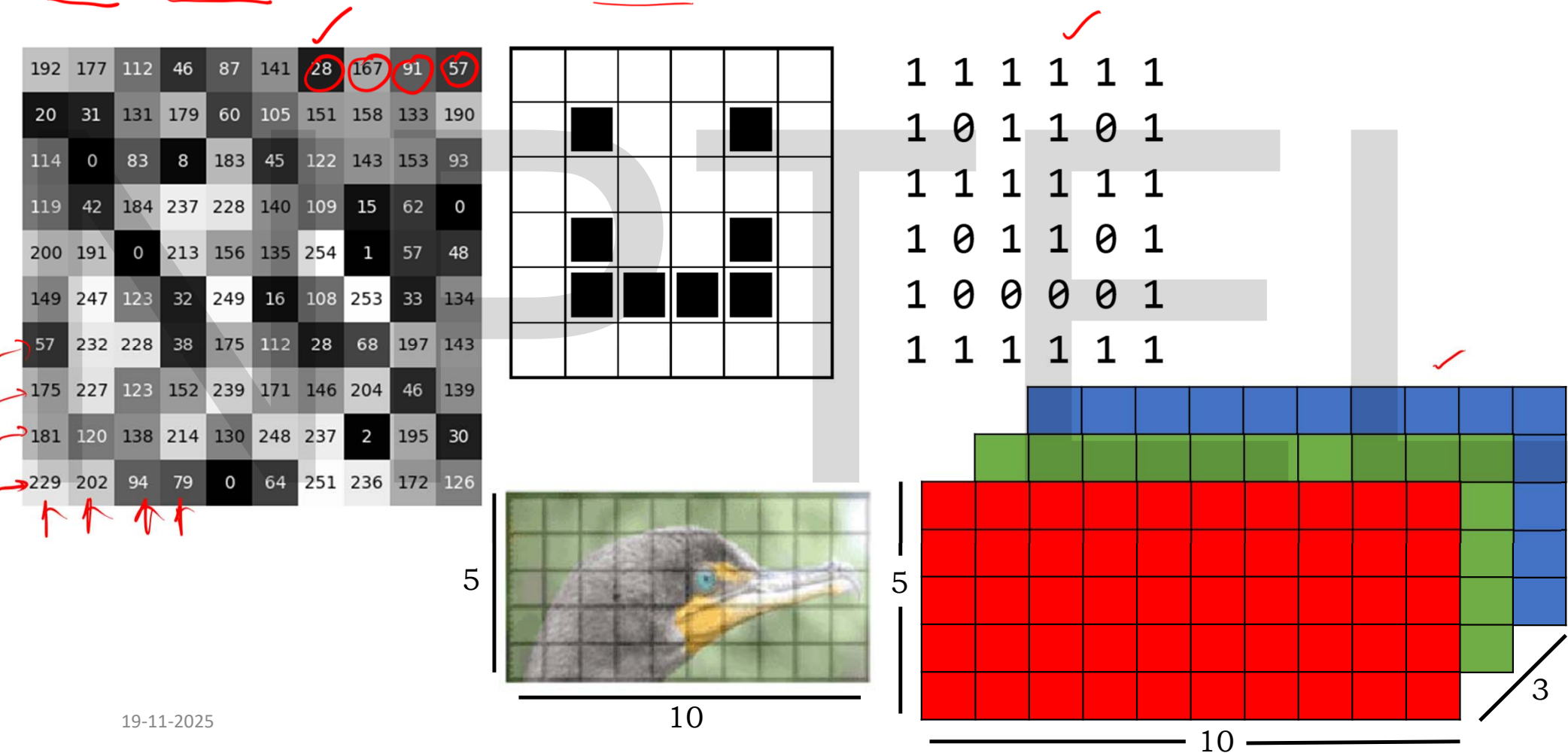
How does the system read an image?

Each pixel in an image is represented by a numerical value. These numerical values are called **pixel values**.

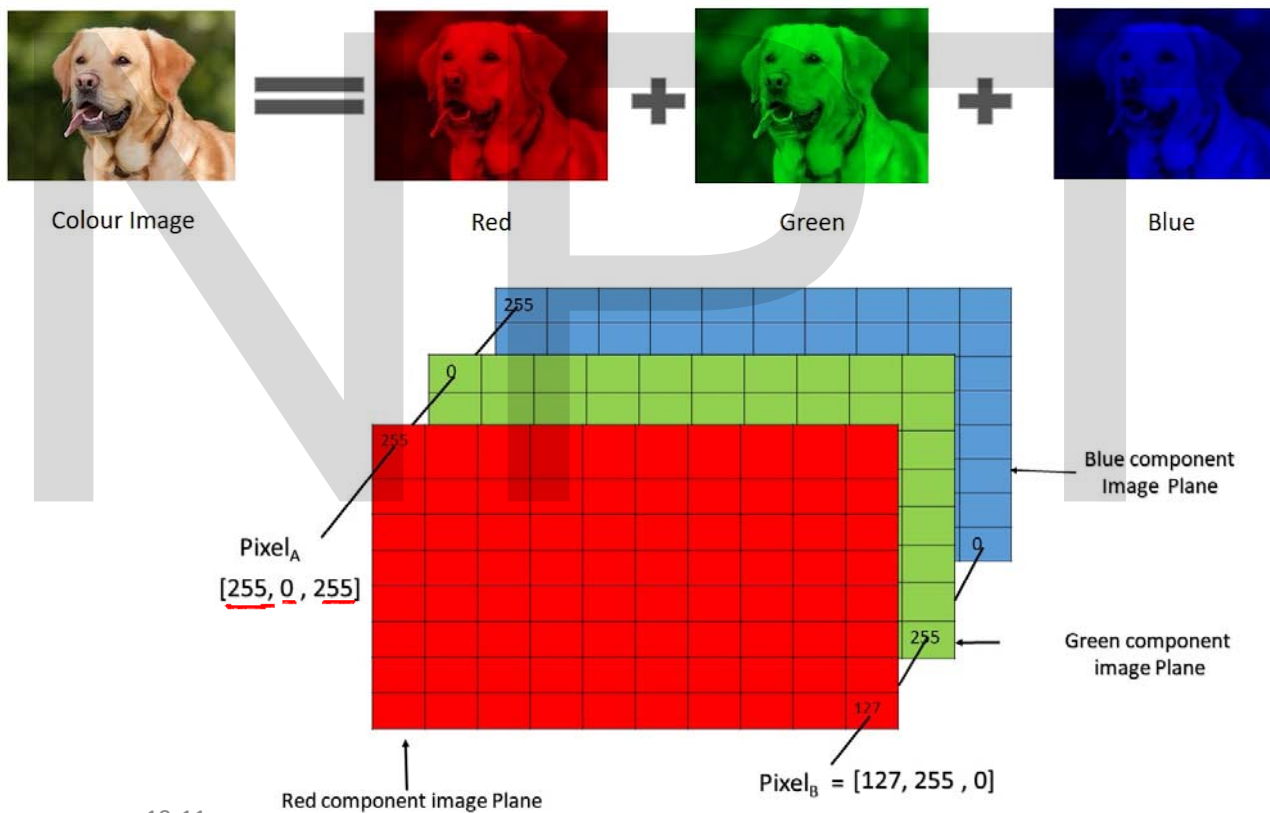


The **dimensions of an image** are defined by the number of pixels along its height and width.

An image can be represented as a **matrix of pixel values**; these pixel values denote the intensity of the pixels (brightness (for grayscale) or color intensity (for RGB)).



Number of channels (Matrix)



Uses Red, Green, and Blue (RGB) components to create any color.

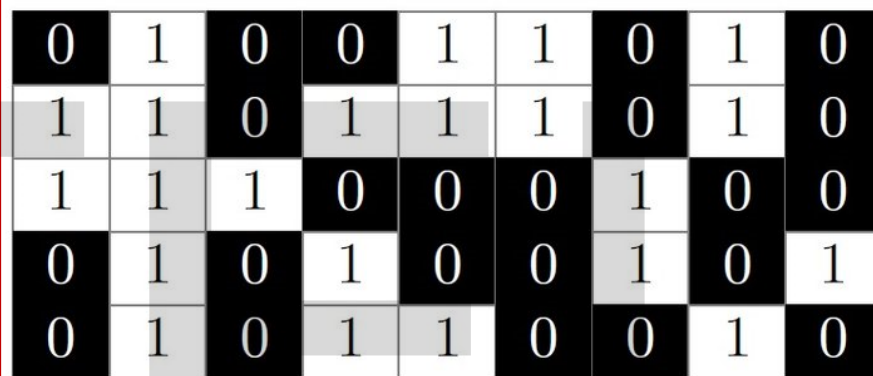
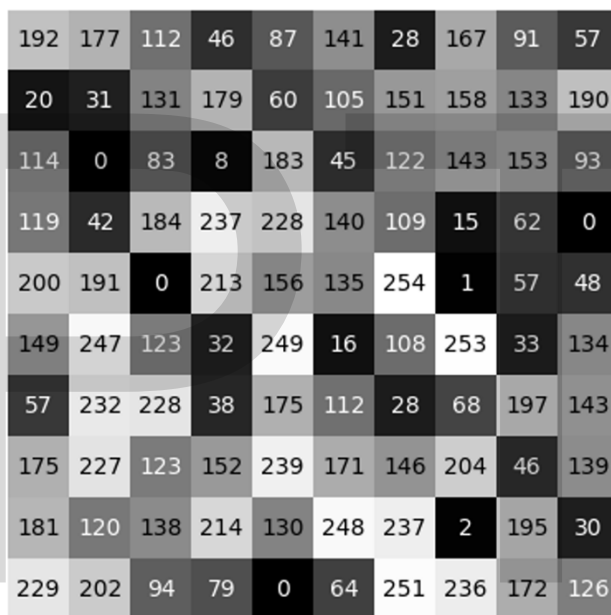
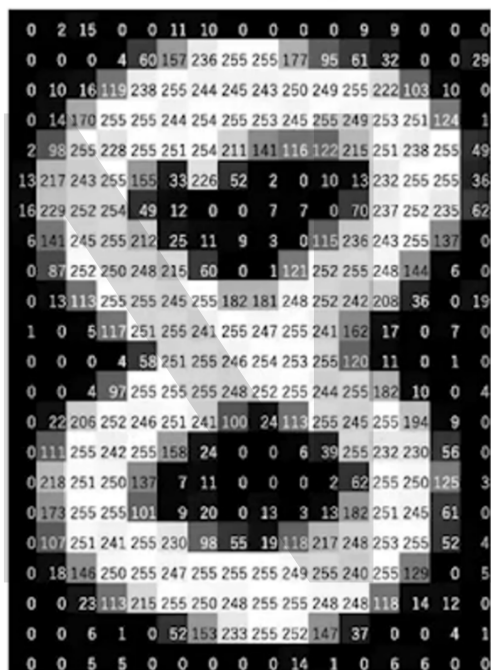
Each pixel is represented by three numerical values — one for each color channel:

Red, Green, and Blue.

Example: $(255, 0, 0) \rightarrow$ pure red.

Since each pixel has three values, an ***RGB image has three channels.***

Number of channels (Matrix)



Binary means the image has only two possible colors: black and white.

Each pixel is represented by a single value — 0 or 1, where

0 → black

1 → white

Since there is only one numerical value per pixel, **Channel = 1**

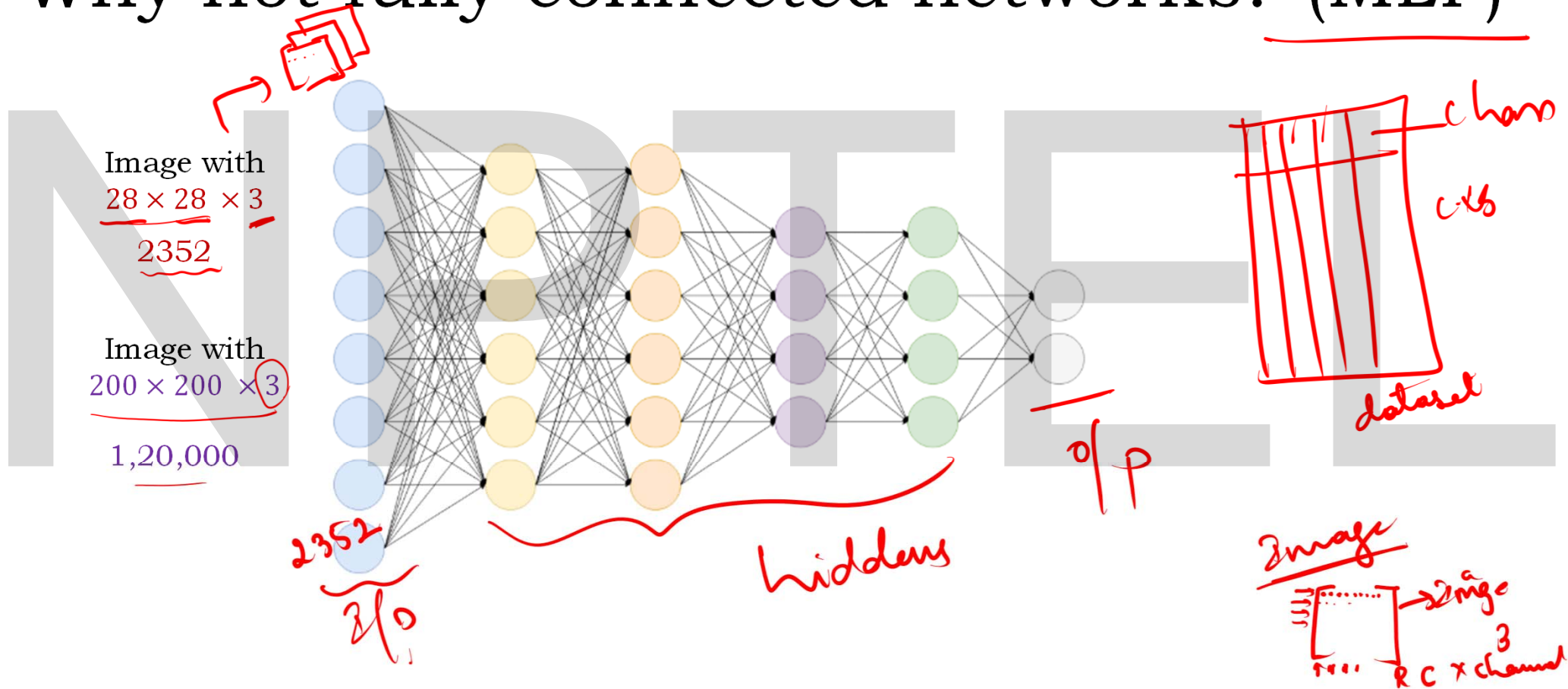
In a grayscale image, **every pixel's value represents how bright or dark it is.**

Since there is only one degree of variation — brightness, **Channel = 1**

Summary

Image Type	Channels	Representation
RGB (Color)	3 ✓	[R,G,B]
Grayscale	1 ✓	Intensity (0-255) ✓
Binary (Black & White) ✓	1 ✓	0 or 1 ✓

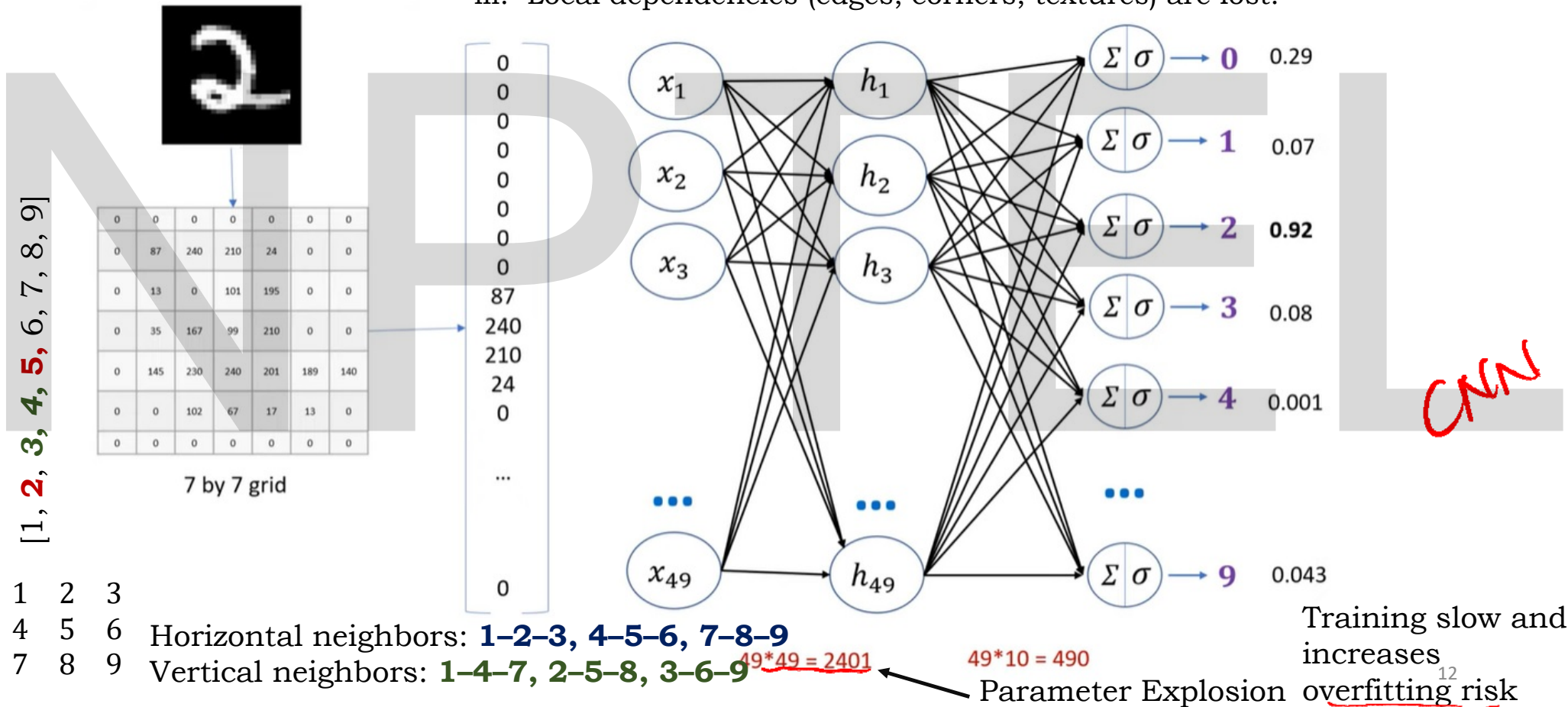
Why not fully connected networks? (MLP)



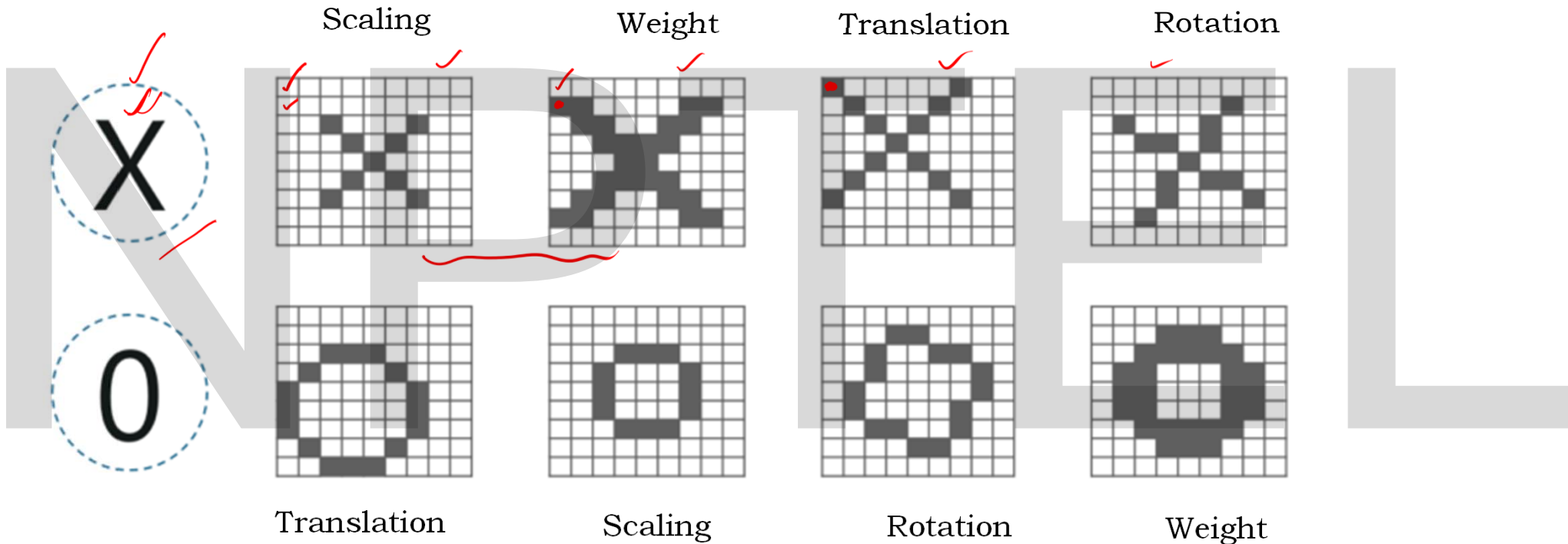
Example

Loss of Spatial Structure:

- Flattening images into 1D input vectors **destroys spatial relationship** (how pixels are positioned and related to each other in space).
- Fully connected NN treats every pixel as an independent feature.
- Local dependencies (edges, corners, textures) are lost.



Data augmentation



In CNNs, **data augmentation** is used to **artificially increase the size** and **diversity of the training dataset** by creating **modified copies of existing images**. This helps the model generalize better and reduces overfitting.

205 101 221 238 222

119 120 230 60 69

168 20 236 90 121

37 116 94 222 111

49 153 102 16 56

0.803 0.396 0.866 0.933 0.870

0.466 0.471 0.902 0.235 0.271

0.659 0.078 0.925 0.353 0.475

0.145 0.455 0.369 0.871 0.435

0.192 0.600 0.400 0.063 0.220

Image of size 5×5

rescale = $1./255$

$$\frac{205}{255}$$

$$\frac{119}{255}$$

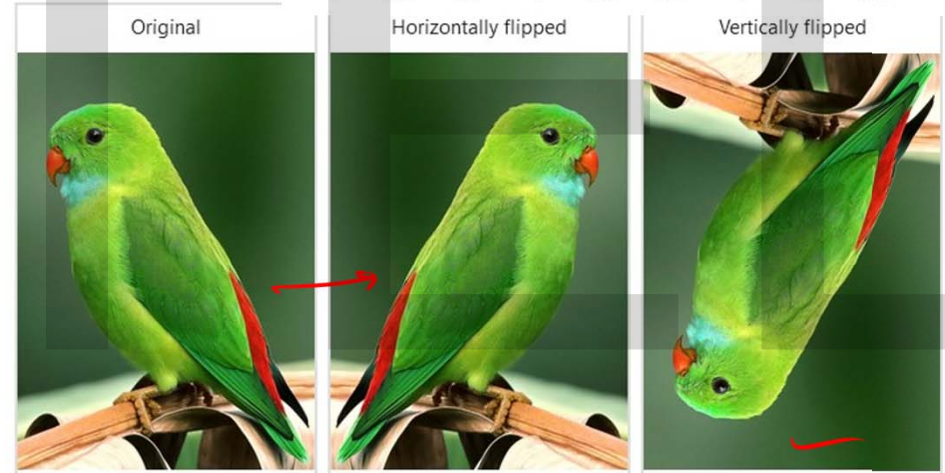
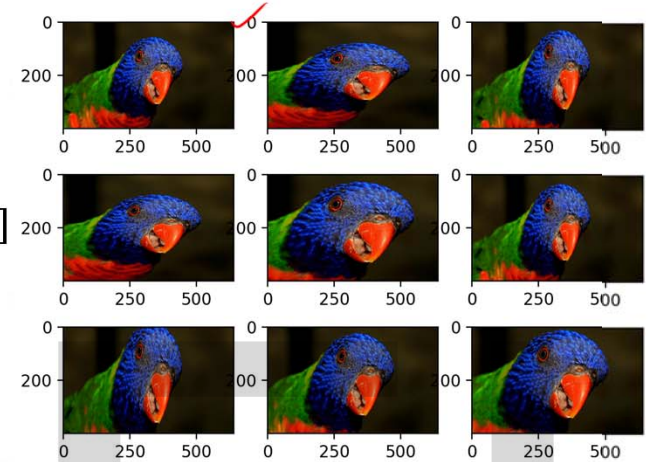
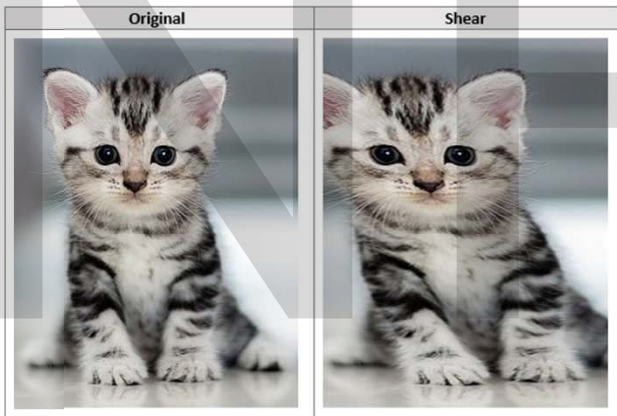
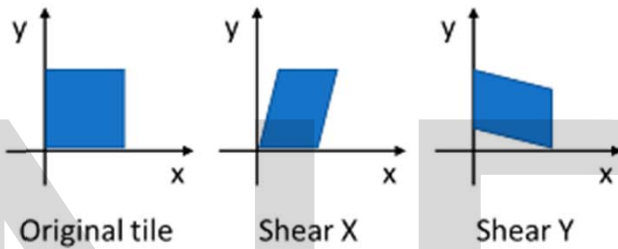
$$\frac{1}{255}$$

Data augmentation

Zoom_range: Helps the model learn to recognize objects at different scales.
 $range = [1 - value, 1 + value]$

Slants the shape of an image along one axis.

$$0.2 \text{ radians} = 0.2 \times \frac{180}{\pi}$$



```
from tensorflow.keras.preprocessing.image import ImageDataGenerator
train_datagen = ImageDataGenerator(rescale = 1./255, shear_range = 0.2, zoom_range = 0.3,
vertical_flip=True, horizontal_flip = True)
test_datagen = ImageDataGenerator(rescale = 1./255)
```

19-11-2025

Source: From Jason Brownlee, *Machine Learning Mastery*
— <https://machinelearningmastery.com/>

width_shift_range: tells how much an image can move **horizontally (left or right)**

Example: width_shift_range = 0.2

$shift \in [-0.2W, +0.2W]$, where $W = \text{width (in pixel) of the image}$.

Suppose if $W = 200$ pixels

Maximum shift = $0.2 \times 200 = 40 \text{ pixels}$

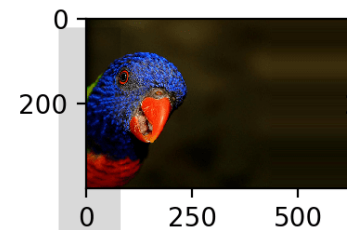
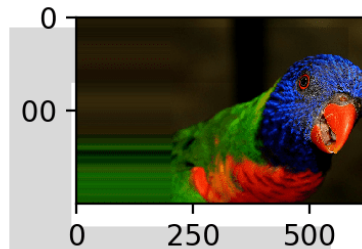
So the image may be shifted:

- **Left** by up to **40 px** (if the random shift is negative), or
- **Right** by up to **40 px** (if positive).

The entire image slides to the right by 40 pixels. The leftmost 40-pixel strip (which now has no image data) becomes blank or filled (based on fill_mode).

The rightmost 40 pixels of the original image go outside the visible frame and get cropped.

$$A = \begin{bmatrix} 1 & 2 & 3 & 4 & 5 \\ 6 & 7 & 8 & 9 & 10 \\ 11 & 12 & 13 & 14 & 15 \\ 16 & 17 & 18 & 19 & 20 \\ 21 & 22 & 23 & 24 & 25 \end{bmatrix} \begin{bmatrix} 1 & 1 & 2 & 3 & 4 \\ 6 & 6 & 7 & 8 & 9 \\ 11 & 11 & 12 & 13 & 14 \\ 16 & 16 & 17 & 18 & 19 \\ 21 & 21 & 22 & 23 & 24 \end{bmatrix} \begin{bmatrix} 0 & 1 & 2 & 3 & 4 \\ 0 & 6 & 7 & 8 & 9 \\ 0 & 11 & 12 & 13 & 14 \\ 0 & 16 & 17 & 18 & 19 \\ 0 & 21 & 22 & 23 & 24 \end{bmatrix}$$



Keras fills that empty space depending on fill_mode:

'nearest' → copies nearby pixel colors

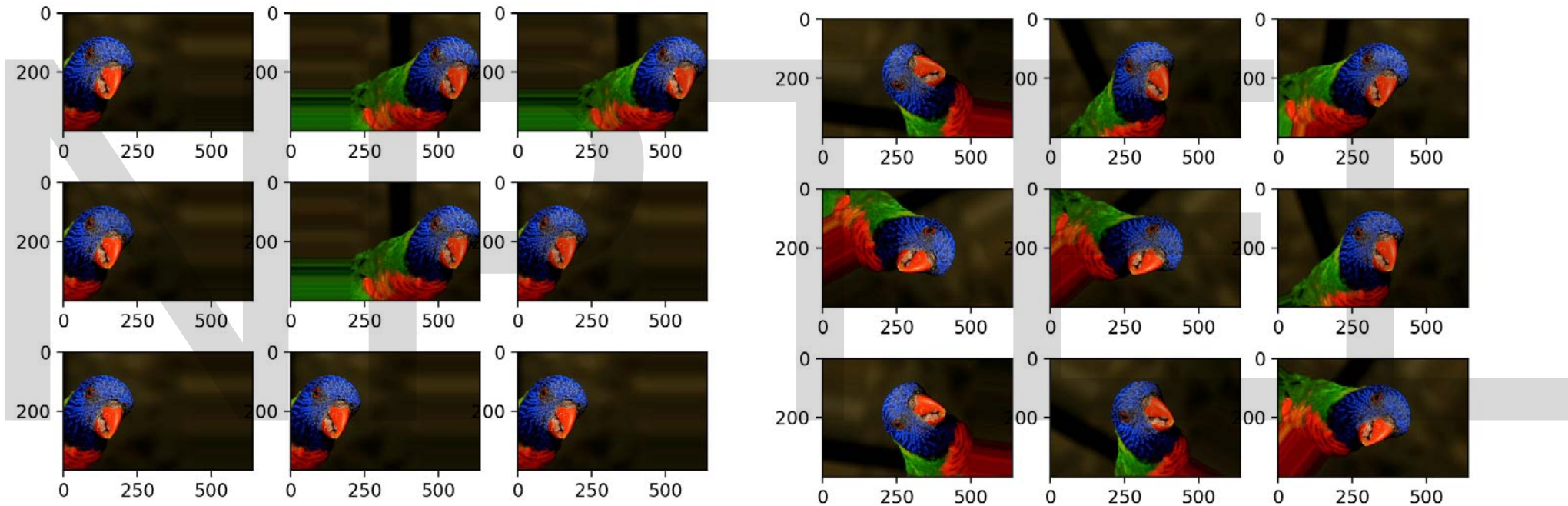
'constant' → fills with a constant color (default: black)

'wrap' → wraps pixels from the other side

'reflect' → mirrors the image

$$\begin{bmatrix} 2 & 1 & 2 & 3 & 4 \\ 7 & 6 & 7 & 8 & 9 \\ 12 & 11 & 12 & 13 & 14 \\ 17 & 16 & 17 & 18 & 19 \\ 22 & 21 & 22 & 23 & 24 \end{bmatrix} \begin{bmatrix} 5 & 1 & 2 & 3 & 4 \\ 10 & 6 & 7 & 8 & 9 \\ 15 & 11 & 12 & 13 & 14 \\ 20 & 16 & 17 & 18 & 19 \\ 25 & 21 & 22 & 23 & 24 \end{bmatrix}$$

Data augmentation



```
from tensorflow.keras.preprocessing.image import ImageDataGenerator  
train_datagen = ImageDataGenerator(width_shift_range=0.2 (shifts image left/right),  
height_shift_range=0.5 (shifts image up/down), rotation_range=90, fill_mode='nearest')  
test_datagen = ImageDataGenerator(rescale = 1./255)
```

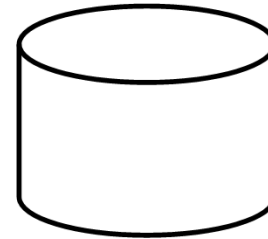
- go to
+ 90°

19-11-2025

Source: From Jason Brownlee, *Machine Learning Mastery*
— <https://machinelearningmastery.com/>

(0.6, 1.4)

```
train_datagen = ImageDataGenerator(rescale = 1./255,  
shear_range = 0.2, zoom_range = 0.4, vertical_flip=True,  
horizontal_flip = True, width_shift_range=0.2 (shifts image  
left/right), height_shift_range=0.5 (shifts image up/down),  
rotation_range=90)
```



Batch = 16

7

Training dataset (112 images)

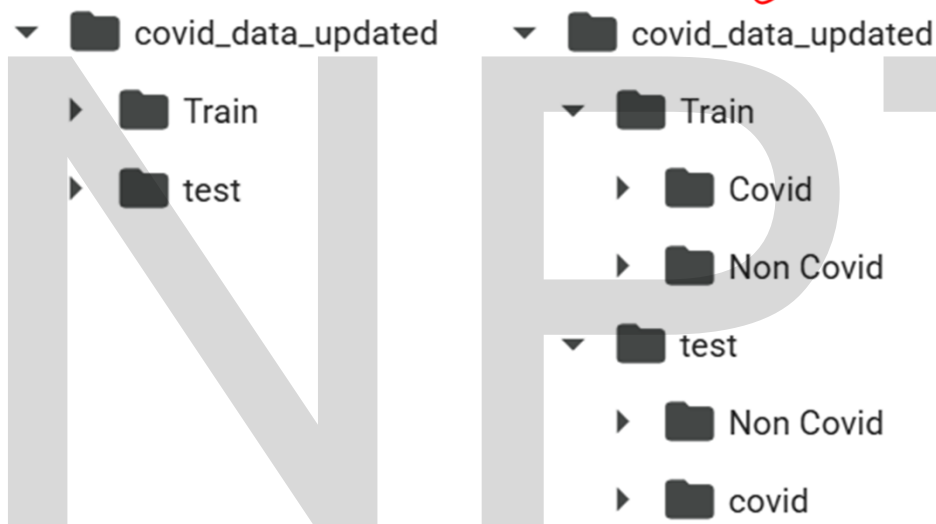
1	2	3	4	5	6	7
1	1	1	1	1	1	1
2	2	2	2	2	2	2
3	3	3	3	3	3	3
4	4	4	4	4	4	4
5	5	5	5	5	5	5
⋮	⋮	⋮	⋮	⋮	⋮	⋮
16	16	16	16	16	16	16

Epoch = 1

50

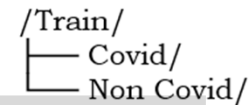
Image	shear_range	zoom_range	vertical_flip	horizontal_flip	width_shift_range	height_shift_range	rotation_range
1	+0.12	1.2×	False	True	+0.18	-0.42	+90°
2	-0.08	0.8×	True	False	-0.11	+0.22	-73°
3	+0.05	1.1×	True	True	+0.06	-0.35	+44°
4	+0.18	0.9×	False	False	-0.03	+0.41	-12°
5	-0.09	1.3×	False	True	+0.19	+0.11	+65°
6	+0.07	1.0×	True	False	-0.08	-0.27	-28°
7	+0.11	0.7×	True	True	+0.15	+0.39	+31°
8	-0.05	1.4×	False	False	-0.17	-0.44	-83°
9	+0.13	1.2×	True	True	+0.04	+0.05	+19°
10	-0.02	0.6×	True	False	-0.20	-0.33	-51°
11	+0.09	1.1×	False	True	+0.10	+0.49	+77°
12	+0.04	0.9×	True	False	-0.06	-0.12	-9°
13	-0.11	1.3×	True	True	+0.08	+0.29	+56°
14	+0.15	0.7×	False	False	-0.12	-0.19	-38°
15	+0.06	1.0×	True	True	+0.03	+0.46	+84°
16	-0.10	1.4×	False	False	-0.09	-0.27	-25°

How to store the data



This is an instance of ImageDataGenerator, It handles image preprocessing and augmentation.

Expect the folder at 'path'. Each subfolder is treated as a class label, and all images in that folder are labeled accordingly.



Keras automatically labels the images based on the subfolder names:

Example:

- All images in Covid/ get label 0
- All in Non Covid/ get label 1 (or vice versa, depending on alphabetical order)

```
training_set = train_datagen.flow_from_directory(
    'path/to/your/data',
    target_size=(height, width),
    batch_size= 8 or 16 or 32,
    class_mode='binary' or 'categorical'
)
```

Loads 8 / 16 / 32 images at a time into memory during training

This function resizes all input images to height and width (e.g., 64×64 or 32×32 or 512×512 pixels), regardless of their original size. This is required because CNNs need a consistent input shape.

Summary

- In today's session, we explored the **fundamentals of image representation** and understood how images are stored as grids of pixel values — whether grayscale, binary, or color images with multiple channels.
- We discussed **why fully connected neural networks are not ideal for image data**, mainly because they destroy spatial relationships between pixels and increase training complexity.
- Finally, we saw how **data augmentation** are performed using ImageDataGenerator in Keras and how to **load and store images** efficiently using flow_from_directory() in Keras.

Overall, this session laid the **foundation for understanding CNN components and the data pipeline.**

Next Session

- In the upcoming session, we will explore the **architecture of Convolutional Neural Networks (CNNs)** in depth.
- The discussion will cover the **key components and layers of CNNs**, including **convolutional layers, pooling layers, activation functions, and fully connected layers**.
- We will analyze how each layer contributes to **feature extraction and hierarchical representation** in images, forming the foundation for modern deep learning models.

NIPTEEL

Thank you